

Sessions as the Agent Substrate

Per-second billing is a poor fit for chat, where the session is mostly silence. It is close to the right fit for an agent, whose idle time is the work.

The problem

An agent turn is long-lived, serial and tool-heavy, and its gaps are the agent working rather than a human pausing, so it is the one workload a per-second clock does not overcharge. Morpheus already bills in that unit and carries tool calls over the session, and holds no trajectory across either.

The mechanism

A trajectory alternates phases $(g_1, t_1, \dots, g_n, t_n)$: g_i seconds generating at $\bar{r}$ tokens per second, t_i seconds blocked on a tool, $G = \sum g_i$, $T = G + \sum t_i$, $\rho = G/T$. A per-token price κ charges $\kappa \bar{r} G$ and ignores every t_i ; a per-second bid p charges pT , an effective $p/(\bar{r}\rho)$ per token. They agree at $\rho^* = p/(\kappa \bar{r})$: chat sits far below it, an agent burst against local tools near $\rho = 1$, and per-second is worse exactly where ρ collapses, forty seconds on a slow external tool buying forty seconds of GPU price and no work.

It is an admission problem before a pricing one, and admission is factored well already. `HasCapacity(modelID)` is the whole interface (`capacity_manager.go:12`): simple counts open sessions, `idle_timeout` ones active inside a hardcoded fifteen minutes (`:73`), the right instinct at the wrong resolution. `PromptActivity` carries `SessionID`, `StartTime`, `EndTime` (`structs.go:32`), enough to seat a second consumer in the slot a parked one leaves idle. One more case in `CreateCapacityManager` (`:112`).

What a trajectory needs that a session is not

Tools run on the provider. `CallAgentTool(ctx, sessionID, agentID, ...)` (`ai_engine.go:195`) runs a registered agent's `command` (`agents_config.go:32`) as an MCP stdio child of the provider process (`:224`). The provider executes the tool on its own credentials, paid per second for as long as it takes, and the consumer authorises by holding a session without seeing what ran. A provider running slow tools is paid more: admission and settlement are one question.

A session spans neither a second bid nor its own funding. History keys on `chat_id`, a client header randomised when absent (`controller_http.go:217`); tools and payment key on `sessionID = H(user, provider, bidId, nonce)` (`SessionRouter.sol:159`), so a trajectory wanting two models holds two sessions whose tool state cannot join although their `chat_id` does. And `endsAt` is fixed at open by `stipend/p` (`:168`) with no extend, so outrunning the funding resumes the task under a new `sessionID`, history intact, tool handles gone.

One client-side object closes both: a `chat_id` and a set of live `sessionIDs`, opened before the current expires rather than after, one per bid, presenting one tool namespace across them. Latency under a per-second bid *is* cost, so the policy falls out: hold the cheap session open across waits, spend into the expensive one only while generating.

Morpheus gets

- Admission on generating load, more consumers per GPU at an unchanged bid, and the first measured ρ , a figure absent from all 62 repositories.
- An agent story on the marketplace. Today it is seven forks: five Vercel-derived templates set an OpenAI base URL to `api.mor.org/api/v1` and never touch a session, while `sim-studio-morpheus` and `Perplexica-morpheus` carry the name and nothing else.

Hanzo and Lux get

- A settled, disputable market in the unit agents consume, and a live price for Enso's cost multiplier.

First step, and who takes it

Hanzo writes the third capacity manager against `dev` and publishes ρ , generate against paid seconds, over its own provider's traffic. No facet upgrade, no 5-of-9 signature, and the figure is the network's either way.

What this does not solve

A trajectory makes provider-side tool execution cheaper to buy and no easier to verify: nothing attests that the tool ran, as nothing attests that the advertised model answered.